

# Polymorphism

Compile time polymorphism

- method overloading already done

runtime polymorphism

- true polymorphism
  - method overriding

## Advantages

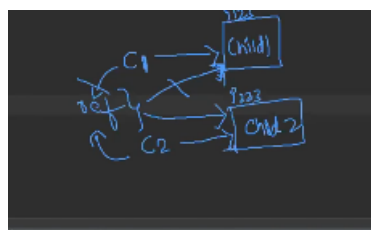
- flexibility
- code size reduces

if we are using the same object type with reference of whatever object class is created such type of programming is called tight coupling

- the first principle for achieving polymorphism is that reference variable type should be of parent class it is called loose coupling example is below
- however we can change the object type for reference the reference type should be of parent type

```
22 public class LaunchParent
23 {
24
25     public static void main(String[] args)
26     {
27
28         Child1 c1=new Child1();
29         Parent1 p1=new Parent1();
30         Child2 c2=new Child2();
31
32         Parent1 ref;
33         ref=c1;
34         ref.cry();
35
36         ref=c2;
37         ref.cry();
38     }
39 }
```

the ref variable was first pointing to the child 1 object then later it was pointing to the child 2 object this is one to many concept shown by one variable



- example of polymorphism

```

class Plane
{
    public void takeOff()
    {
        System.out.println("Plane is taking off");
    }
    public void fly()
    {
        System.out.println("Plane is flying");
    }
    public void landing()
    {
        System.out.println("Plane is landing");
    }
}
class CargoPlane extends Plane
{
    public void fly()
    {
        System.out.println("CargoPlane flies at lower hieght");
    }
}
class PassengerPlane extends Plane
{
    public void fly()
    {
        System.out.println("PassengerPlane flies at medium height");
    }
}
class FighterPlane extends Plane
{
    public void fly()
    {
        System.out.println("Fighter plane flies at high height");
    }
}

class Airport
{
    public void permit(Plane plane)
    {
        plane.takeOff();
        plane.landing();
    }
}

```

```

        plane.fly();
    }
}

public class LaunchPlane {
    public static void main(String[] args) {

        CargoPlane cp=new CargoPlane();

        PassengerPlane pp=new PassengerPlane();

        FighterPlane fp=new FighterPlane();

        Airport a=new Airport();
        a.permit(cp);
        a.permit(pp);
        a.permit(fp);

//        Plane plane;
//
//        plane=cp;
//        plane.takeOff();//1:M
//        plane.fly();
//        plane.landing();
//
//        plane=pp;
//
//        plane.takeOff();//1:M
//        plane.fly();
//        plane.landing();

    }
}

```

- Point to be notes that using parent type reference we cannot call directly the specific method of the child however we can call the inherited and overridden method of the class child
- however we can call it through indirect method shown below
- for child class object creating parent type is called upcasting while temporarily assigning the ref type to child type is called down casting

```

}
public class LaunchParent2 {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Parents p=new Child11();
        p.cry();
        //p.eat(); // directly using parent type ref you can not
        ((Child11) p).eat();//downcasting
        Parents p2=new Child12();
        p2.cry();
    }
}

```

## ABSTRACTION

IT REFERS TO THE PROCESS OF HIDING THE ACTUAL IMPLEMENTATION ONLY FEATURES ARE THERE

- SOME KEY POINTS
- abstract keywords are used with methods
- IN JAVA WE CAN HAVE METHOD WITHOUT IMPLEMENTATIONS/body HOWEVER THOSE METHODS MUST BE DECLARE abstract
- in one class if even one method is also abstract then class must be declared as abstract too
- abstract class can have both abstract and normal methods we cant apply abstract keyword to a variable
- if u will try to inherit the abstracted class then the methods of abstraction will also come along with it and will throw error inorder to avoid this u have to implemment the abstraction methods of the parent class in child class that is u needdd to override them or u need to declare child class as well abstract
- you can not create object of abstract class
- constructor cant be made as abstract ;

### abstract class is incomplete class and child class will implemented

however we should not make a abstracted class as final because its an illegal combination we cant inherit a final class it will be of no use

- final method can be inherited to another class however we cant override it so this coombination is also illegal
- we can have constructor in abstract class;;;;;

## small project on all oops concept

```

import java.util.Scanner;

abstract class area
{

```

```

abstract void input();
abstract float compute();
abstract void display();

}

class computingArea{

    void computing(area ar)
    {
        ar.input();
        ar.display();
    }

}

class Rectangle extends area
{
    float length;
    float breadth;
    Scanner sc=new Scanner(System.in);
    void input()
    {
        length =sc.nextFloat();
        breadth =sc.nextFloat();

    }
    float compute()
    {
        return length*breadth;

    }
    void display()
    {
        System.out.println("the area is "+compute());

    }

}

class Circle extends area
{
    float radius;
    final float pi= 3.14F;

```

```

Scanner sc=new Scanner(System.in);
void input()
{
    radius  =sc.nextFloat();

}
float compute()
{
    return pi*radius*radius;

}
void display()
{
    System.out.println("the area is "+compute());

}

}

class Square extends area
{
    float side;
    Scanner sc=new Scanner(System.in);
    void input()
    {
        side  =sc.nextFloat();

    }
    float compute()
    {
        return side*side;

    }
    void display()
    {
        System.out.println("the area is "+compute());

    }

}

class power{

```

```
public static void main(String[] args) {  
    Rectangle rectangle=new Rectangle();  
    Square square=new Square();  
    Circle circle=new Circle();  
    computingArea computingArea=new computingArea();  
    computingArea.computing(square);  
    computingArea.computing(rectangle);  
    computingArea.computing(circle);  
  
}  
  
}
```

Consider below code of Test.java file:

```
public class Test {  
    public static void main(String[] args) {  
        for(int i = 5; i >= 1; i--) { //Line n1  
            System.out.println("*".repeat(i)); //Line n2  
        }  
    }  
}
```

What will be the result of compiling and executing Test class?

A. \*  
\*\*  
\*\*\*  
\*\*\*\*  
\*\*\*\*\*

B. \*\*\*\*\*  
\*\*\*\*  
\*\*\*  
\*\*  
\*

C. \*  
\*\*  
\*\*\*  
\*\*\*\*

D. \*\*\*\*\*  
\*\*\*\*  
\*\*\*  
\*\*  
\*

E. compilationError

F. Line n2 cause runtimeerror

Answer : B

Q>

```
public static void main(String... args) {  
    boolean opt=true;//line5  
    switch (opt){  
        case true://line7  
            System.out.print("True");  
            break;//line9  
        default:  
            System.out.println("*****");  
    }  
    System.out.println("Done");  
}
```

What modification should be enabled to print TrueDone?

A. Replace line 5 with String opt="True"  
Replace line 7 with case "True".

B. Replace line 5 with boolean opt=1  
Replace line 7 with case 1=

C. At line 9 remove break statement

D. Remove the default section.

Answer: A

Q>

```
public static void main(String... args) {  
    int x=5; //line4  
    while (isAvailable(x)){//line5  
        System.out.println(x);//line6    JVM:: x-- , first use x and then  
x= x-1    :: 54321  
        //line7  
    }  
}  
public static boolean isAvailable(int x){  
    return x-- > 0 ? true :false;//line8  
}
```

What modification should be enabled the code to print 54321

- A. Replace line 6 with System.out.print(x--);
- B. At line 4 insert x--
- C. Replace line 6 with --x and line 7,insert System.out.print(x)
- D. Replace line 8 with return (x>0) ? false:true

Answer: A

Q>

```
public static void main(String... args) {  
    StringBuilder sb=new StringBuilder(5);  
    String s="";  
    if (sb.equals(s)){  
        System.out.println("Match 1");  
    }else if(sb.toString().equals(s.toString())){  
        System.out.println("Match 2");  
    }else{  
        System.out.println("Match 3");  
    }  
}
```

- A. Match 1
- B. Match 2
- C. CompileTime Error
- D. Match3
- E. NullPointerException

Answer: B

Q>

Consider below code of Test.java file:

```
public class Test {  
    public static void main(String [] args) {  
        String text = "RISE ";  
        text = text + (text = "ABOVE ");  
        System.out.println(text);  
    }  
}
```

What will be the result of compiling and executing Test class?

- A. RISE RISE ABOVE
- B. RISE ABOVE
- C. ABOVE ABOVE
- D. RISE ABOVE RISE

Answer: B

Q>

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Java");  
        String s1 = sb.toString();  
        String s2 = "Java";  
        System.out.println(s1 == s2);  
    }  
}
```

- A. compilation error
- B. true
- C. false
- D. An exception is thrown at runtime

Answer: C

Q>

```
public class Test {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Java");  
        String s1 = sb.toString();  
        String s2 = sb.toString();  
        System.out.println(s1 == s2);  
    }  
}
```

- A. compilation error
- B. true
- C. false
- D. An exception is thrown at runtime

Answer: C

Q>

```
public class Test {  
    public static void main(String[] args) {  
        String str = "java";  
        StringBuilder sb = new StringBuilder("java");  
        System.out.println(str.equals(sb) + ":" + sb.equals(str));  
    }  
}
```

- A. Compilation Error
- B. false:false
- C. false:true
- D. true:false
- E. true:true

Answer: B

Q>

A bank's swift code is generally of 11 characters and used in international money transfers.

An example of swift code: ICICINBBRT4

ICIC: First 4 letters for bank code

IN: Next 2 letters for Country code

BB: Next 2 letters for Location code

RT4: Next 3 letters for Branch code

Which of the following code correctly extracts country code from the swift code referred by String reference variable swiftCode?

- A. swiftCode.substring(4,6);
- B. swiftCode.substring(5,6);
- C. swiftCode.substring(5,7);
- D. swiftCode.substring(4,5);

Answer: ICICINBBRT4

A. swiftCode.substring(4,6);

Q>

```
public class Test {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder();  
        System.out.println(sb.append(null).length());  
    }  
}
```

- A. NullPointerException
- B. 1
- C. 4
- D. CompilationError

append(String)  
append(StringBuilder)  
append(StringBuffer)

Answer: D

Q>

```
public class Test {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder();  
        System.out.println(sb.append("").append("").append("").length());  
    }  
}
```

- A. 0
- B. 1
- C. 2
- D. 3

Answer: A(0)

Q>

```
public class Test {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder(5); // capacity = 5 , new capacity =  
(5+1) * 2 = 12  
        sb.append("0123456789");  
        sb.delete(8, 1000);  
        System.out.println(sb);  
    }  
}
```

- A. CompilationError
- B. An Exception is thrown at Runtime
- C. 01234567
- D. 89

Answer: C

```
Q>
public class Test {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hurrah! I Passed...");
        sb.delete(0, 100);
        System.out.println(sb.length());
    }
}
A. 19
B. 0
C. 16
D. StringIndexOutOfBoundsException
```

Answer: B

```
Q>
public class Test {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder(100);
        System.out.println(sb.length() + ":" + sb.toString().length());
    }
}
What will be the result of compiling and executing Test class?
A. 100:100
B. 100:0
C. 16:16
D. 16:0
E. 0:0
```

Answer: 0:0

